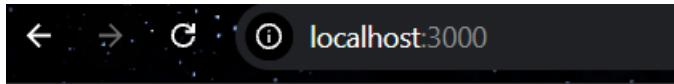


Actividad 3
Diseño de backend con
reglas de negocio
y control de estados

Abish Abril Santana García
Desarrollo Full Stark
Universidad Tecmilenio

Servidor funcionando

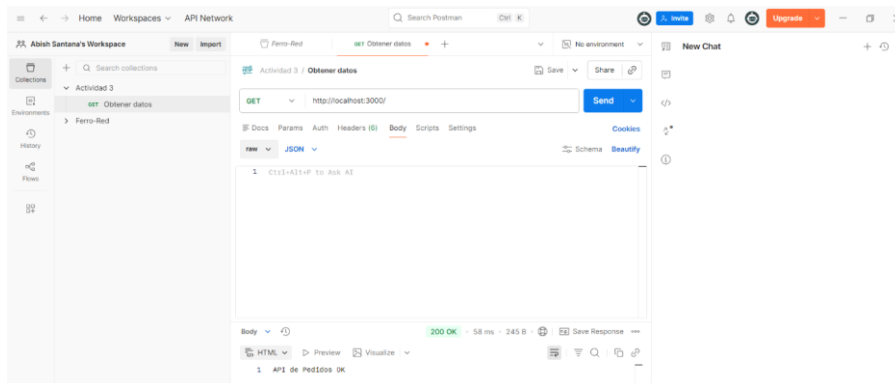
```
PS C:\Proyecto\proyecto-pedido> node index.js
Servidor corriendo en http://localhost:3000
```



Pedidos OK

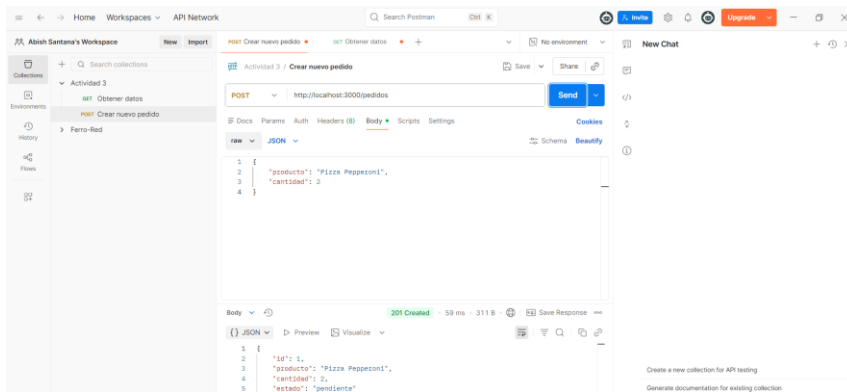
Creación del método Get

- <http://localhost:3000/>



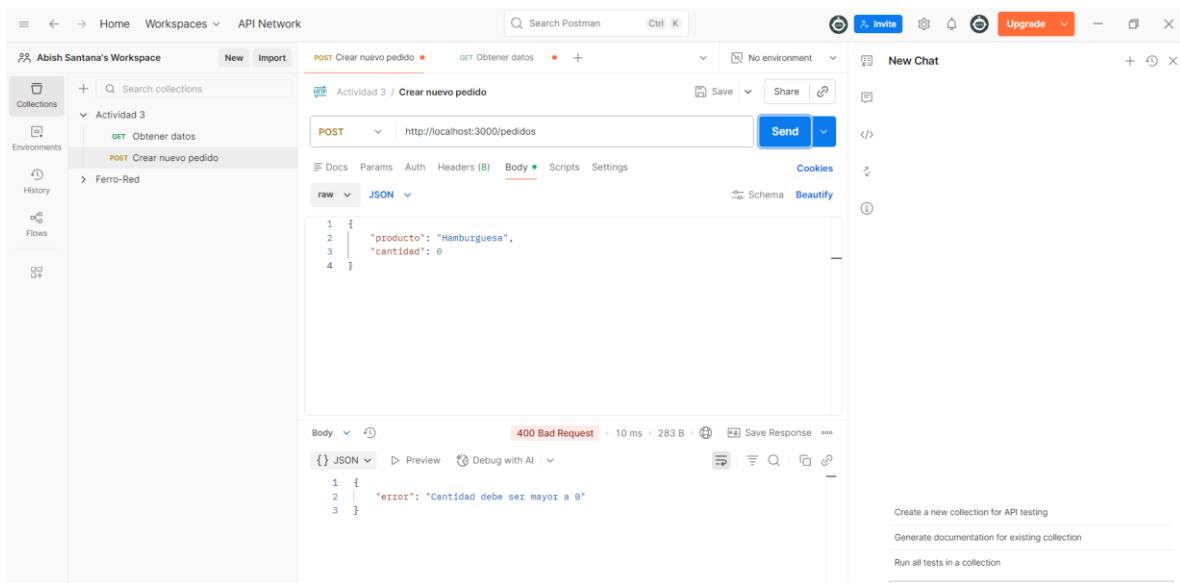
Crear pedido exitoso (201)

- <http://localhost:3000/pedidos>



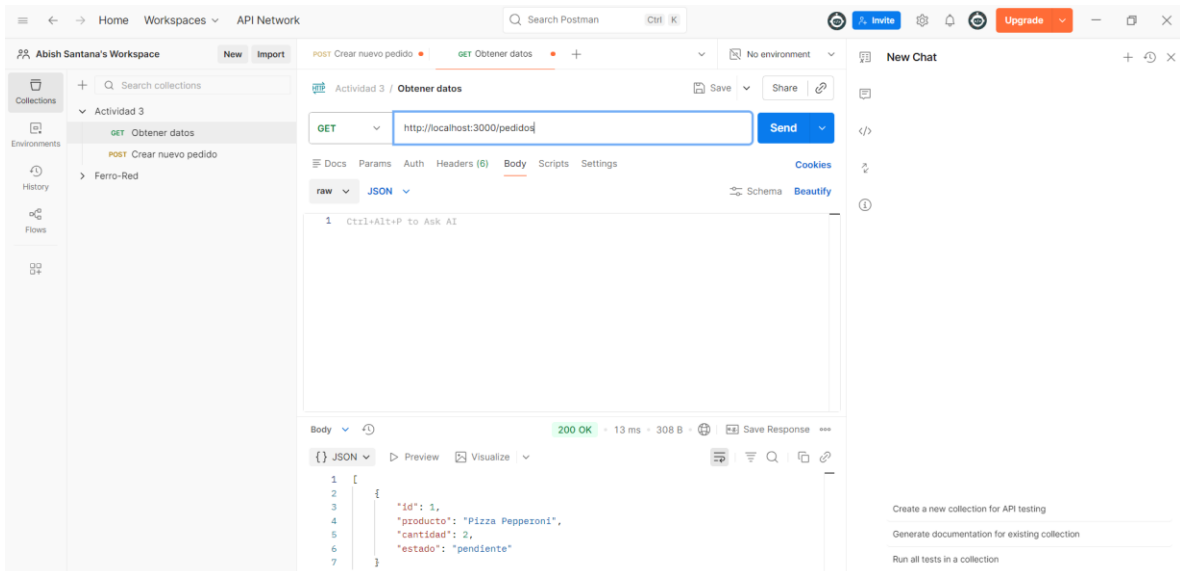
Crear pedido con error (cantidad 0)

- <http://localhost:3000/pedidos>



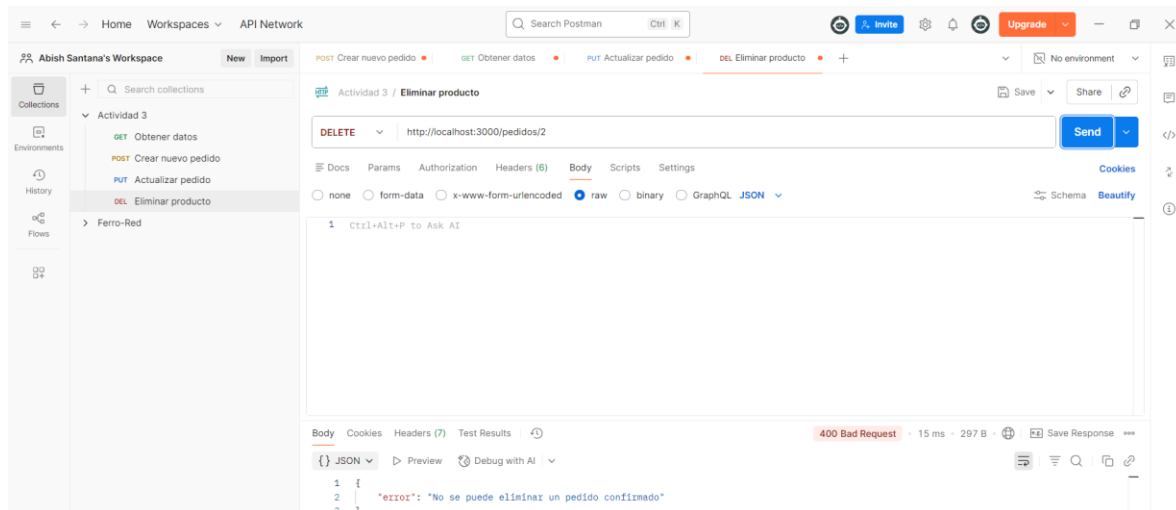
Ver todos los pedidos

- <http://localhost:3000/pedidos>



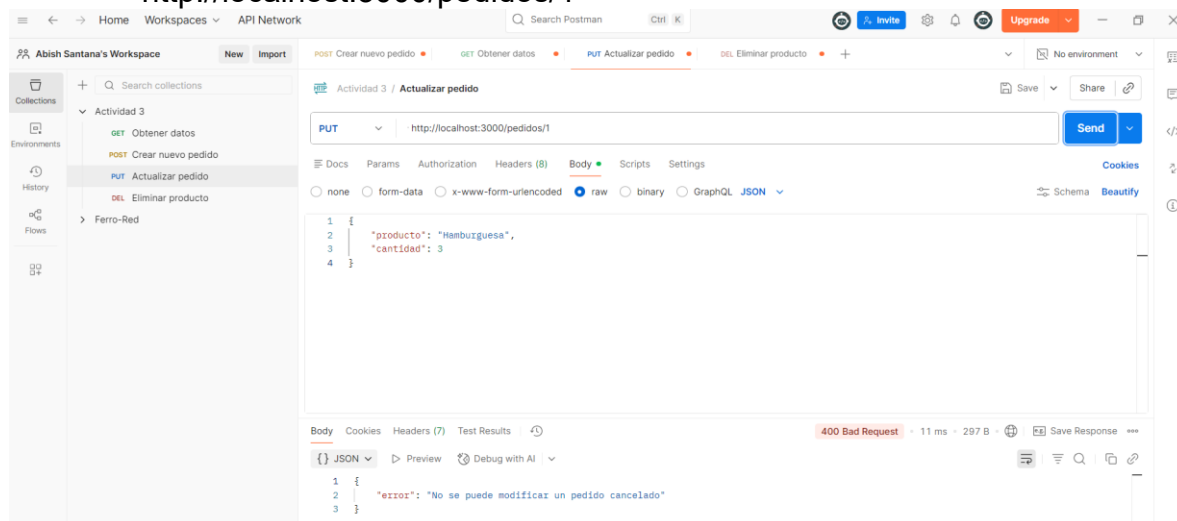
Intentar eliminar pedido confirmado (error)

- <http://localhost:3000/pedidos/2>



Cancelado no puede modificarse

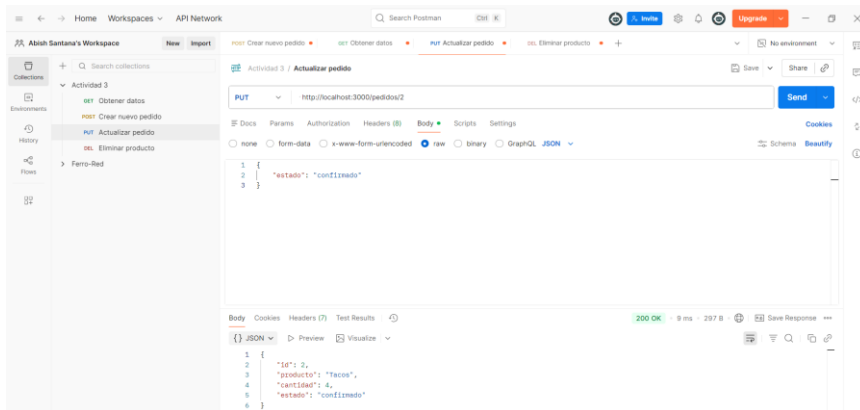
- <http://localhost:3000/pedidos/1>



Solo el estado de pendiente puede cambiar a otros estados

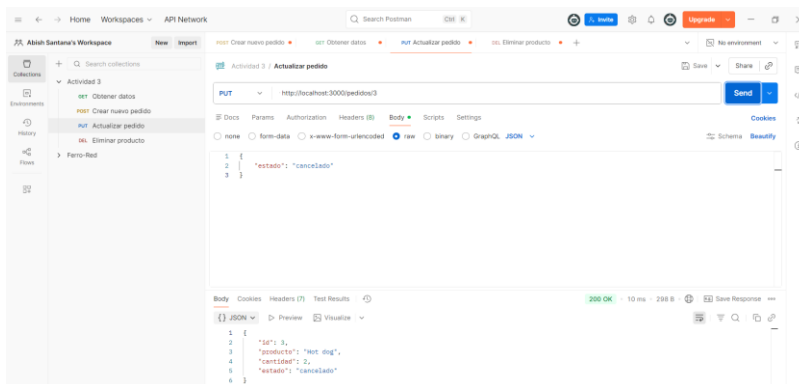
- Estado de confirmado

```
1 {  
2   "id": 2,  
3   "producto": "Tacos",  
4   "cantidad": 4,  
5   "estado": "pendiente"  
6 }
```

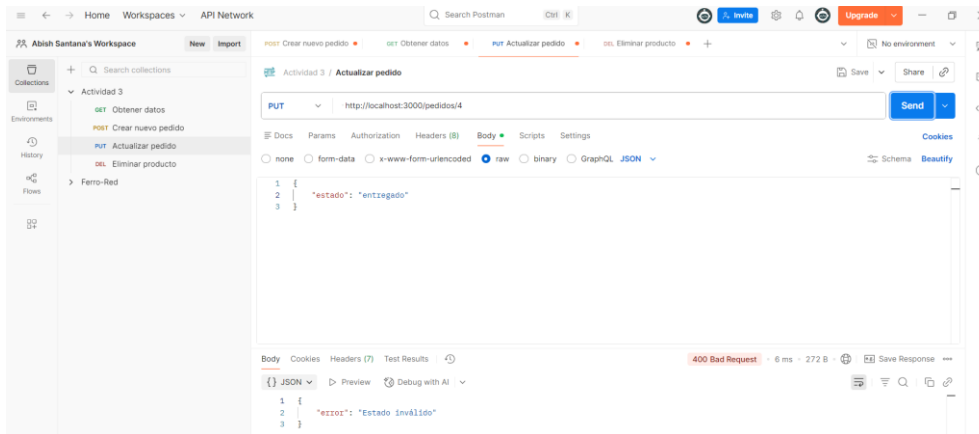


- Estado de cancelado

```
1 {  
2   "id": 3,  
3   "producto": "Hot dog",  
4   "cantidad": 2,  
5   "estado": "pendiente"  
6 }
```



Estados inválidos

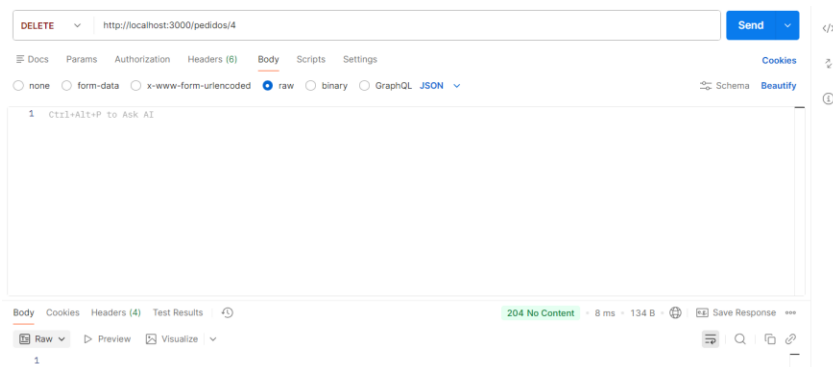


Eliminar pedido pendiente

Body Cookies Headers (7) Test Results

{ } JSON Preview Visualize

```
1 {  
2   "id": 4,  
3   "producto": "Salchi papas",  
4   "cantidad": 2,  
5   "estado": "pendiente"  
6 }
```



Profe las capturas no coinciden con los datos que se guardaron en la documentación porque realicé cambios y cuando volví a llamar los datos se me olvido que se borraban y volví a realizar las pruebas, pero tienen diferentes id y datos.

1. ¿Qué reglas se implementaron?

Reglas del profe implementadas:

1. Estado inicial "pendiente" en todo pedido nuevo
2. No se aceptan cantidades 0 o negativas
3. Solo pedidos "pendientes" se pueden modificar
4. Pedidos "confirmados" y "cancelados" NO se modifican
5. Solo pedidos "pendientes" se pueden eliminar

Reglas extras implementadas:

6. Validación de producto no vacío
7. Solo 3 estados permitidos: "pendiente", "confirmado", "cancelado"
8. Encapsulación de datos con variables privadas #
9. Mensajes de error claros con códigos HTTP específicos

2. ¿En qué archivos están las reglas?

- Repository (pedidos.repository.js)

Ahí van las reglas de cómo debe funcionar el sistema por dentro.

Por ejemplo, la regla de solo pedidos pendientes se puede cambiar va ahí porque es una regla del negocio, no del usuario.

```
class PedidosRepository {  
  #pedidos;  
  #nextId;
```

```
  if (pedido.estado !== "pendiente") {  
    return false;  
  }
```

```
  const nuevoPedido = {  
    id: this.#nextId++,  
    producto: producto,  
    cantidad: cantidad,  
    estado: "pendiente"
```

- Controller (pedidos.controller.js)

Ahí van las reglas de lo que el usuario puede o no mandar.

Por ejemplo, la cantidad debe ser mayor a 0 va ahí porque es algo que se revisa en cuanto llega la petición, antes de hacer nada más.

Se separan así para no revolver responsabilidades:

- El repository se encarga de la lógica.
- El controller se encarga de validar lo que llega.

```
const cantidadNumber = Number(cantidad);
if (cantidadNumber <= 0) {
  return res.status(400).json("error: Cantidad debe ser mayor a 0");
}

const nuevo = repo.create(producto, cantidadNumber);
return res.status(201).json(nuevo);
```

```
// REGLA 5: Solo actualiza si está "pendiente"
if (pedidoExistente.estado !== "pendiente") {
  return res.status(400).json(
    "error: No se puede modificar un pedido" + pedidoExistente.estado
  );
}

// Validar el estado si se quiere cambiar
if (req.body.estado && ![ "pendiente", "confirmado", "cancelado" ].includes(req.body.estado)) {
  return res.status(400).json({ error: 'Estado inválido' });
}

// Validar cantidad si se quiere cambiar
if (req.body.cantidad && req.body.cantidad <= 0) {
  return res.status(400).json("error: Cantidad inválida");
}

const actualizado = repo.update(id, req.body);

if (actualizado === false) {
  return res.status(400).json("error: No se puede actualizar");
}

if (!actualizado) {
  return res.status(404).json("error: No encontrado");
}

return res.json(actualizado);
}

function remove(req, res) {
  const id = Number(req.params.id);
```

```
// Primero verificar que existe
const pedido = repo.getById(id);
if (!pedido) {
  return res.status(404).json("error: Pedido no encontrado");
}

// REGLA 6: Solo elimina si está pendiente
if (pedido.estado !== "pendiente") {
  return res.status(400).json("error: No se puede eliminar un pedido" + pedido.estado);
}

const eliminado = repo.delete(id);

if (eliminado === false) {
  return res.status(400).json("error: No se puede eliminar");
}

if (!eliminado) {
  return res.status(404).json("error: No encontrado");
}

return res.status(204).send();
}

module.exports = { getAll, getById, create, update, remove };
```


3. ¿Por qué tomaste estas decisiones?

Separación de responsabilidades:

- Routes solo direccionan peticiones
- Controller maneja todas las validaciones y reglas de negocio
- Repository se encarga del almacenamiento con datos protegidos

Encapsulación con #:

- Variables privadas protegen los datos del acceso directo
- Previene modificaciones no autorizadas
- Sigue la estructura de poo

Validaciones en controller:

- Centraliza todas las reglas de negocio en un solo lugar
- Facilita cambios futuros en las reglas
- Proporciona mensajes de error claros al usuario

Códigos HTTP específicos:

- 201 para creación exitosa
- 400 para errores de validación
- 404 para recursos no encontrados
- 204 para eliminación exitosa

Dividir archivos por carpetas

- Cambiar reglas solo requiere modificar el controller
- Fácil agregar nuevas funcionalidades o cambio.

Link documentación

<https://documenter.getpostman.com/view/51907016/2sBXc8pj1h>